



**NANYANG
TECHNOLOGICAL
UNIVERSITY**
SINGAPORE

NTU Academy for Professional
and Continuing Education

Agents & Skills

Build and deploy the configuration files that make an AI agent work the way you need it to

Module 6

Created by :

Swarup Biswas



The opening experiment

The workspace is a folder of ordinary work documents – raw meeting notes, a half-written status report, an email thread, a budget. We are going to ask it the same question four times, adding exactly one configuration file between rounds: first a context file, then a rule, then a skill.

Build-along: the four rounds

The rhythm for every round: I run it on screen; you run the same round from the walkthrough handout; we check the checkpoint together before moving on.

- **Round 1** — no configuration: tidy formatting, missing knowledge (initials, vague dates, mixed currency).
- **Round 2** — AGENTS.md: real names, real dates, house style.
- **Round 3** — a rule: open questions lead, in every response.
- **Round 4** — a skill: the full minutes structure, one owner per action, and an honest "decisions made: none".

The mental model

You have just used all four configuration layers with your own hands. This part gives you the map, and the decision framework for choosing between them.

You configure a harness, not a model

These tools do not expose model settings such as temperature or fine-tuning. They expose files that decide what each conversation is given.

The same model answers like a stranger in an unconfigured project and like a colleague in a configured one — the difference is entirely in the files.

Because the files describe your project and your working rules rather than the AI itself, they survive model upgrades and even a change of vendor.

Four layers shape every conversation

Layer	When it loads	Who decides	File
Context	Every conversation	Nobody — always on	`AGENTS.md`
Rules	Every conversation	Nobody — always on	`agents/rules/`
Skills	When the task matches	The agent	`agents/skills/`
Commands	When you invoke them	You	`agents/workflows/`

Two questions separate the layers: when does it load, and who decides. Skills are the only layer the agent selects for itself.

Skills are the only layer the agent chooses

At startup the agent reads only each skill's name and description — roughly **100 tokens** per skill.

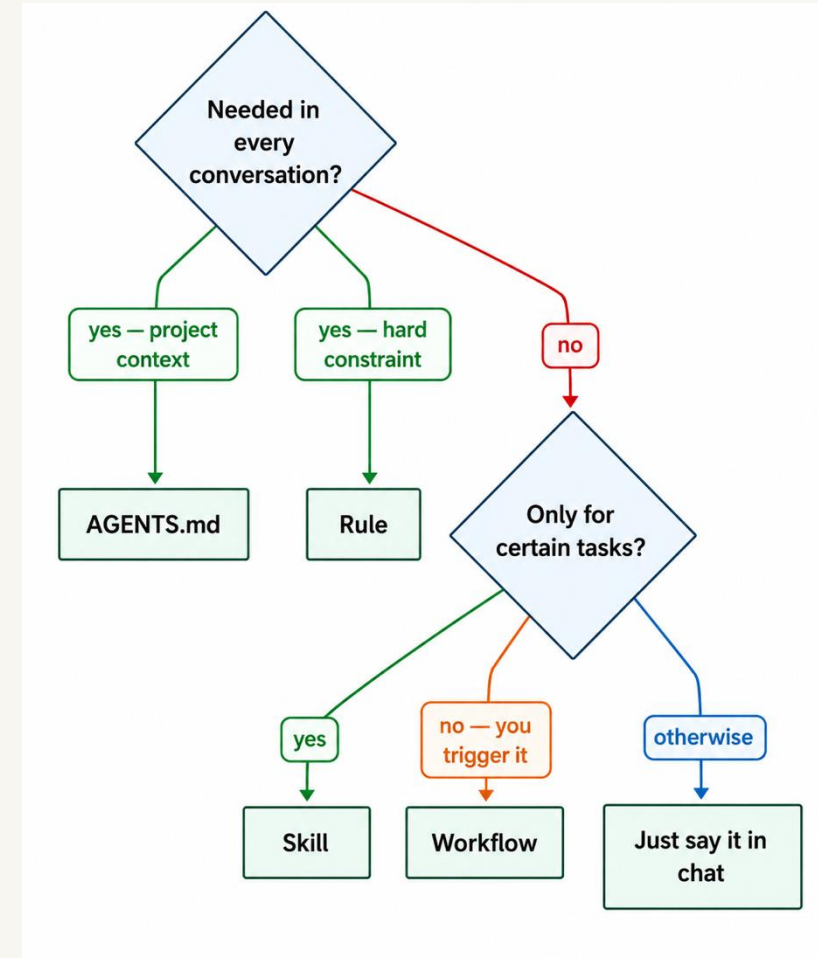
Thirty installed skills therefore cost about 3,000 tokens of awareness, not 150,000 tokens of content.

The full instructions load only when your request matches the description.

This makes the description a router: if it does not match the words you actually type, the skill never runs, however good its instructions are.

The decision framework

If an instruction is needed only once, saying it in the chat is the correct choice — not everything deserves to be codified.



Where your files go: project versus global

Scope	Context	Skills
This project — travels with the folder	`AGENTS.md` at the folder root	`.agents/skills/<name>/SKILL.md`
Global, Antigravity — follows you	`\${HOME}/.gemini/AGENTS.md`	`\${HOME}/.gemini/config/skills/`
Global, Zed — follows you	`\${HOME}/.config/zed/AGENTS.md`	`\${HOME}/.agents/skills/`

Project files serve the team and travel with the folder; global files follow you. Team truths go in the project; personal gap compensations go global. Rules and workflows follow the same pattern inside `.agents/`.

Source: vendor documentation, accessed 11 July 2026

What the evidence says about context files

In a field study across 124 pull requests, a well-written AGENTS.md made agents **28.6% faster** at the median, because the file cut wasted codebase exploration.

A separate evaluation found that context files raised inference cost by **20%** with no improvement in success rates — when the files carried generic project descriptions.

The resolution: explicit instructions are followed and pay for themselves; overview prose costs tokens and changes nothing.

Source: [arXiv:2601.20404](#); [arXiv:2602.11988](#)

Four ways to give an agent knowledge

Method	What it gives the agent	Shape
MCP	Tool access — what it can reach, not when to use it	Live connections to services
RAG	Factual knowledge pulled at run time	Retrieved chunks from a knowledge base
Fine-tuning	Knowledge baked into the model's weights	Permanent, expensive retraining
Skills	Procedural knowledge — how, in what order, with what judgement	Markdown under version control

These complement rather than compete: a skill can tell the agent when to call a tool and what to do with what it retrieves. In human terms, RAG is semantic memory, chat logs are episodic memory, and skills are procedural memory — knowing how.

Source: Xu & Yan, *arXiv:2602.12430*; Anthropic engineering, Oct 2025

Why a skill and not a longer prompt

The model has read more than any of us ever will — most of the public web, text and code. What it has never read is how your team works: your report format, your document standards, the fifteen instructions behind your monthly status pack.

That missing piece is **procedural knowledge** — and it is why a very capable model still cannot produce your report correctly without being told, every time.

Option one is prompting the fifteen steps each month: the result drifts with your wording, and two people with different prompting styles get two different documents.

Option two is writing the procedure down once, as a skill — a procedure the team keeps, versions, and improves.

A skill is a folder with a SKILL.md

```
yaml
---
name: meeting-minutes      # matches the folder name
description: >
  Turns raw meeting notes into structured minutes. Use when
  the user asks for minutes, or says "tidy these notes"
  or "what did we decide?"
---
```

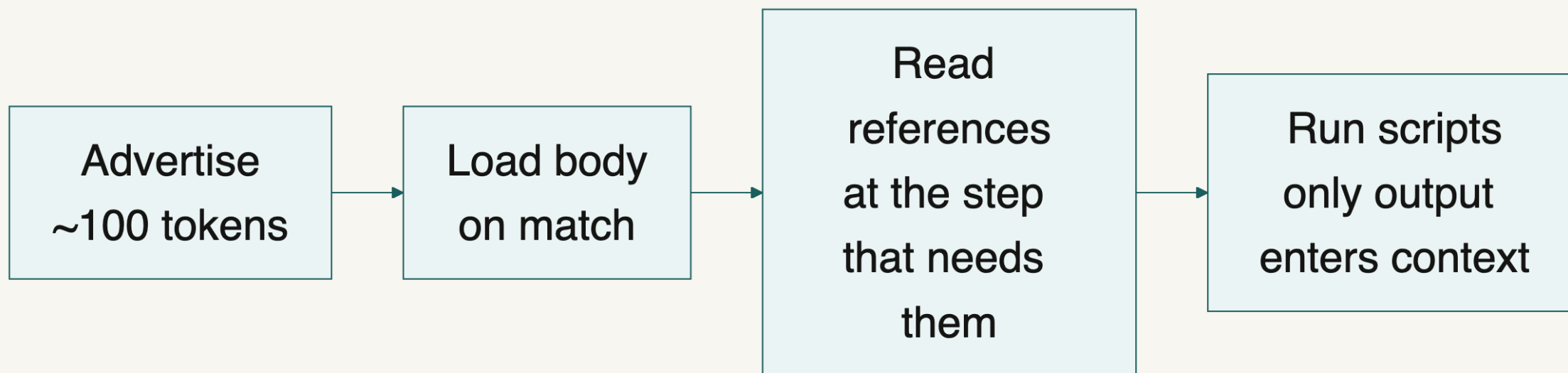
The frontmatter is what the agent sees at startup; everything below it — numbered steps, a NEVER list, exit criteria — loads only on activation. The file lives at ``.agents/skills/meeting-minutes/SKILL.md``, and that path matters later when we look at portability.

The description is a router, not a label

Weak	Strong
"A skill for meeting documents"	"Turns raw notes into structured minutes with decisions and action items. Use when the user asks for minutes, a summary, or says 'tidy these notes'."

The agent matches the words you type against the description, so write it like search keywords rather than documentation: state what the skill does, then list the phrases a person actually uses — including the casual ones, because you are the person.

Progressive disclosure keeps a large skill library affordable



Each stage loads only on demand, so you pay for what runs rather than what is installed. The practical rule: keep the SKILL.md body lean and move templates and long detail into a `references/` folder, which the agent reads only at the step that needs it.

Three guardrails keep the agent on the instructions

A **NEVER list** states what the agent must not do while the skill is active. Prohibitions hold better than positive suggestions, which agents quietly reinterpret.

Exit criteria are observable yes-or-no checks that define "done", so the agent neither stops early nor keeps generating past the point of usefulness.

Anti-rationalisation lines pre-write the counter to the agent's predictable excuse: when it decides "this case is different", your file already says it is not.

Build all three from shortcuts you actually catch the agent taking, not from imagination.

Which configuration travels to which tool – July 2026

Tool	AGENTS.md	`.agents/skills/`
Antigravity	Native	Native
Zed	In a fallback chain	Native
Gemini CLI	Via one setting	Native
Claude Code	Bridge file needed	Copy to `.claude/skills/`
Cursor / Copilot / Codex	Native	Not documented

The skill folder now travels better than the context file, because a skill carries its own trigger – each tool can adopt the format without agreeing on precedence rules. The honest edges: Claude Code needs a one-line bridge file, and in Zed a leftover `.cursorrules` silently wins over AGENTS.md.

Source: vendor documentation, accessed 11 July 2026



Skills are code someone else wrote

A skill is instructions your agent will follow, and scripts it may run. Installing one is an act of trust.

The risk is real: in the first marketplace audit (February 2026), **341 of 2,857** skills were malicious — 824 two weeks later — delivering credential-stealing malware through a "paste these commands" prerequisites section.

The named threats: prompt injection hidden in the instructions, tool poisoning through self-granted permissions, and malware in `scripts/`.

The habits: install only from sources you trust, read the SKILL.md **and its scripts** before enabling, and prefer version-controlled team copies over live marketplace installs.

Source: Koi Security, Feb 2026; Xu & Yan, arXiv:2602.12430

Version Management

Month/Year	Version	Changes
May 2026	0.0	New Module Design
July 2026	1.0	Improvement to the content and updated information